

Informe de Testing — Sprint 2

Introducción

Durante el Sprint 2 se desarrollaron tests automatizados para el backend del sistema de gestión de turnos. El objetivo establecido era alcanzar una cobertura mínima del 80% del código. Al finalizar el sprint se logró una cobertura global del **84%**

Herramientas utilizadas

- **JUnit 5**: framework principal para la escritura y ejecución de tests.
 - **Mockito**: librería para crear mocks (objetos falsos) de las dependencias, permitiendo testear clases en aislamiento sin necesidad de base de datos ni servicios externos.
 - **MockMvc**: herramienta de Spring para simular requests HTTP en los tests de integración, sin necesidad de levantar un servidor real.
 - **H2**: base de datos en memoria utilizada exclusivamente durante los tests de integración. Reemplaza a PostgreSQL en el entorno de test, garantizando que cada ejecución parte de un estado limpio.
 - **JaCoCo**: plugin de Maven utilizado para medir la cobertura de código.
-

Tipos de tests realizados

Tests unitarios

Testean una clase en aislamiento, sin levantar Spring ni tocar la base de datos. Las dependencias externas se reemplazan por mocks. Son muy rápidos y permiten verificar la lógica de negocio de forma precisa.

Se realizaron tests unitarios sobre las siguientes clases:

UserService — lógica de usuarios y sesiones:

- Login con credenciales correctas, password incorrecta y usuario inexistente.
- Verificación de roles en el token devuelto al hacer login.
- Registro de nuevo cliente y nuevo profesional.
- Intento de registro con email ya existente (debe devolver vacío y no llamar a save).
- Verificación de que al registrarse se llama a save exactamente una vez.
- Refresh token válido, vencido e inexistente.
- Verificación de que el refresh token se elimina al usarse (uso único).
- Recuperación de contraseña con email existente e inexistente.
- Cambio de contraseña con credenciales correctas e incorrectas.

- Verificación de roles implícitos: CLIENTE, PROFESIONAL y SUPER_ADMIN.

Reserva — entidad de dominio:

- Estado inicial PENDIENTE al crear una reserva.
- Verificación de cliente y turno asociados.
- Transiciones de estado: confirmar() y cancelar() desde distintos estados.

Turno — entidad de dominio:

- Verificación de campos al construir un turno: agenda, cliente, fecha, bloque horario y estado.
- Cambios de estado con setEstado(): CONFIRMADO, DESHABILITADO y OCUPADO_SIN_CONFIRMAR.

Agenda — entidad de dominio:

- Verificación de campos al construir una agenda: profesional y bloqueMinutos.
- Agenda nueva sin rangos horarios.
- Múltiples rangos en distintos días.
- updateConfig(): reemplaza rangos existentes, cambia bloqueMinutos y permite quedar sin rangos.

Servicio — entidad de dominio:

- Verificación de campos al construir un servicio: nombre, duración, precio y agenda asociada.

Tests de integración de controllers

Testean el comportamiento completo del sistema desde el HTTP, incluyendo autenticación, validaciones de DTOs, códigos de respuesta y permisos. Levantan el contexto de Spring con base de datos H2 en memoria. Cada test corre dentro de una transacción que se revierte al finalizar, garantizando aislamiento entre tests.

SessionRestController — /sessions

Endpoint	Escenario testeado	Código esperado
POST /sessions	Login con credenciales correctas	201
POST /sessions	Login con password incorrecta	401
POST /sessions	Login con usuario inexistente	401
POST /sessions	Login con email con formato inválido	400

PUT /sessions	Refresh con token inválido	401
POST /sessions/recover	Recuperación con email existente	200

UserRestController — /users

Endpoint	Escenario testado	Código esperado
POST /users	Registro de nuevo usuario	201
POST /users	Registro con email duplicado	409
POST /users	Registro con fecha de nacimiento futura	400
GET /users/me	Obtener perfil con token válido	200
GET /users/me	Obtener perfil sin token	403
GET /users	Profesional busca clientes	200
GET /users	Cliente intenta buscar otros clientes	403
GET /users	Cliente busca profesionales	200
PUT /users/password	Cambio de contraseña correcto	200
PUT /users/password	Cambio con contraseña actual incorrecta	400
PUT /users/password	Cambio sin token	403

AgendaTurnoIntegrationTest — /agenda y /turno

Seguridad:

- Acceso sin token a agenda y turno devuelve 403.

Agenda — GET /agenda:

- Agenda inexistente devuelve 404.
- Profesional puede ver su propia agenda con todos los datos (rangos, turnos).
- Profesional puede ver su agenda a través de /mi-agenda sin necesitar el ID.
- Cliente no puede acceder a /mi-agenda.
- Cliente puede ver la agenda pública de un profesional
- Agenda pública de profesional inexistente devuelve 404.

Agenda — PUT /agenda:

- Cliente no puede modificar agenda (403).
- Profesional puede actualizar su agenda sin conflictos.
- Actualización sin conflictos actualiza correctamente bloqueMinutos y rangos.
- Actualización con turnos conflictivos devuelve 409 con lista de turnos afectados.
- Actualización con conflictos: al pasar `?conflictos=cancelar` en la URL, los turnos afectados se cancelan automáticamente.
- Actualización con conflictos: al pasar `?conflictos=mantener` en la URL, los turnos se preservan y la agenda se actualiza igual.

Turno — POST /turno:

- Profesional puede crear un turno con cliente asignado (estado OCUPADO_SIN_CONFIRMAR).
- Profesional puede crear un bloque sin cliente (estado DESHABILITADO, ejemplo si se va de vacaciones).
- Cliente puede reservar su propio turno sin enviar su ID (el sistema lo detecta por el token).
- Si el cliente envía un ID ajeno en el body, el sistema lo ignora y usa el ID del token.
- Turno duplicado en mismo bloque horario devuelve 409.
- Turno en agenda inexistente devuelve 404.

Turno — GET /turno:

- Turno inexistente devuelve 404.
- Profesional puede ver turno de su agenda con todos los datos correctos.
- Otro profesional no puede ver turno ajeno (403).
- Cliente puede ver su propio turno.

Turno — PUT /turno:

- Cliente puede confirmar su propio turno.
- Cliente no puede cambiar turno a estado DESHABILITADO (403).
- Profesional puede cambiar turno a cualquier estado (CONFIRMADO, DESHABILITADO, OCUPADO_SIN_CONFIRMAR).
- Actualizar turno inexistente devuelve 404.

Turno — DELETE /turno:

- Profesional puede eliminar turno; el turno ya no existe después.
- Cliente puede eliminar su propio turno (ejemplo se enfermo).
- Eliminar turno inexistente devuelve 404.

Integración agenda-turno:

- Los turnos creados aparecen correctamente en el detalle de la agenda.
-

Cobertura obtenida

Paquete	Cobertura de líneas
agenda	86%
config	85%
turno	81%
user	88%
notificacion	100%
reserva	100%
servicio	100%
Total	84%

Conclusiones

La combinación de tests unitarios y tests de integración resultó complementaria: los tests unitarios permiten verificar la lógica de negocio en detalle y de forma rápida, mientras que los tests de integración garantizan que los endpoints HTTP se comportan correctamente en cuanto a códigos de respuesta, validaciones y permisos.

Un punto importante fue la compatibilidad entre H2 y PostgreSQL. La columna `value` de la entidad `RefreshToken` es una palabra reservada en H2 pero no en PostgreSQL, lo que generaba errores en los tests de integración. Se resolvió configurando `NON_KEYWORDS=VALUE` en el `application.properties` de test, sin afectar el entorno de producción.

Se superó el objetivo de cobertura del 80% establecido por la cátedra, alcanzando un 84% de líneas cubiertas para este sprint.